

백엔드 포트폴리오

dlsfuf0316@gmail.com | github.com/lnryeolChoi | Seoul, South Korea

- API를 단순 엔드포인트가 아니라 사용자 흐름, 데이터 상태, 예외 케이스가 이어지는 구조로 정리합니다.
- 인증, 데이터베이스, 배포 환경처럼 서비스 안정성에 직접 영향을 주는 기반 요소를 함께 다룹니다.
- 직접 구현한 시스템 프로그래밍 프로젝트를 통해 네트워크, 프로세스, 파일 디스크립터, 동시성의 기초를 실전 맥락에서 익혔습니다.

Thread safety

심심조각

Spring Boot backend for an emotion-based diary service

감정 기록 기반 일기 서비스에서 소셜 로그인, 일기 CRUD, AI 편지, 요약, 월간 키워드 리포트 흐름을 하나의 백엔드로 연결했습니다.

문제

소셜 로그인, 토큰 재발급, 일기 데이터 관리, 외부 AI 응답 생성이 서로 끊기지 않고 안정적으로 이어져야 했습니다.

구현

Spring Security와 JWT를 중심으로 인증 흐름을 구성하고, MySQL과 Redis를 연결해 사용자 정보와 토큰 상태를 분리했습니다. AI 응답은 로컬 AI 서버와 연동하고, 응답 형식이 달라지는 예외 상황을 분석해 파싱 로직을 보강했습니다.

결과

인증, 데이터 저장, AI 생성, 월간 조회 흐름을 하나의 백엔드 안에서 관리할 수 있는 구조를 만들었습니다.

- Google, Apple 소셜 로그인과 JWT 재발급 흐름 구현
- 일기 저장, 조회, 수정, 삭제 및 월별 기록 조회 API 구성
- 외부 AI API 응답 구조 차이를 분석하고 테스트 케이스로 검증

사용 기술

Spring Boot	Java	Spring Security	JPA
MySQL	Redis	JWT	

열렬히.뛰기

Go와 SQLite로 만든 학습 기록 블로그

노션과 GitHub에 흩어진 학습 기록 1,400여 편을 이관하고, 공개 화면과 Admin 편집기를 Go 단일 바이너리로 구현했습니다.

문제

이관 중 수식·목록·이미지·내부 링크가 깨질 수 있었고, 이후 재이관이나 배포가 새 글을 덮어쓰지 않아야 했습니다.

구현

이관 오류를 유형별로 전수 검사해 변환 규칙을 고쳤습니다. 사람의 분류·제외·본문 보정은 코드에 기록해 재실행 결과가 수렴하도록 했습니다.

결과

서버 DB를 정본으로 삼아 코드 배포와 DB 교체를 분리하고 백업·업로드 가드를 마련했습니다.

- 노션 블록을 Markdown으로 변환하고 GitHub 글·이미지·내부 링크 이관
- 공개 렌더러를 공유하는 미리보기와 블록 인라인 편집 구현
- 변경 충돌 검사와 재이관 시 수렴하는 보정 규칙 적용

사용 기술

Go	SQLite	Markdown	Content migration
Backup			

ft_irc

Event-driven IRC server

여러 사용자 연결과 채널 상태를 다루는 IRC 서버를 구현하며 네트워크 서버 구조와 이벤트 기반 입출력 방식을 학습했습니다.

문제

여러 클라이언트 연결, 명령 처리, 채널 상태가 동시에 움직이기 때문에 기능을 추가할수록 서버 구조가 쉽게 복잡해질 수 있었습니다.

구현

서버의 큰 책임을 먼저 나누고, kqueue 기반 이벤트 감지 흐름을 중심으로 클라이언트 연결과 채널 상태 처리를 정리했습니다.

결과

이벤트 기반 서버가 연결, 명령, 상태를 어떤 흐름으로 처리하는지 구현 맥락에서 이해했습니다.

- 기능 추가 전 전체 서버 구조를 먼저 나누어 설계
- 멀티플렉싱 기반 논블로킹 I/O 시스템 콜인 kqueue의 이벤트 감지 흐름 이해
- 클라이언트 연결과 채널 상태를 다루는 서버 동작 방식 학습

사용 기술

C++	Socket	kqueue	IRC
Network server			

minishell

Unix shell parsing and process execution

셸이 입력을 해석하고 여러 명령을 연결해 실행하는 과정을 직접 구현하며 파싱과 프로세스 흐름을 익힌 프로젝트입니다.

문제

셸은 입력 문자열을 그대로 실행하는 것이 아니라 파이프, 리다이렉션, 환경 변수 같은 규칙에 맞게 구조화해야 했습니다.

구현

입력이 어떤 단계로 파싱되어 실행 구조로 이어지는지 학습하고, pipe, fork, dup2, execve를 이용해 여러 명령을 연결했습니다.

결과

문자열 파싱, 실행 계획 생성, 프로세스 연결이 한 흐름으로 이어지는 방식을 직접 구현했습니다.

- 입력을 실행 가능한 구조로 바꾸는 파싱 흐름 학습
- 여러 명령을 연결하는 pipe 구조 구현
- 셸 명령 실행 과정과 프로세스 연결 방식 이해

사용 기술

C	Unix process	pipe	fork
dup2	execve		

transcendence

Service-style web backend with authentication and runtime setup

실제 웹 서비스 흐름을 기준으로 인증, 데이터 저장, 실행 환경까지 포함한 백엔드를 구축했습니다.

문제

API 서버, 로그인, 보안, 데이터베이스, 배포용 실행 환경이 함께 맞물려야 했습니다.

구현

Django REST Framework로 API를 구성하고, 42 OAuth 로그인, JWT 발급과 재발급, 이메일 기반 2차 인증 링크를 인증 흐름에 통합했습니다.

결과

인증 실패와 토큰 만료 상황까지 고려한 백엔드 흐름과 PostgreSQL, Docker Compose 기반 실행 환경을 구성했습니다.

- Django + Django REST Framework 기반 백엔드 구축
- 42 OAuth 로그인, JWT 발급, 이메일 기반 2차 인증 링크 구현
- PostgreSQL와 Docker Compose를 이용한 개발 환경 구성

사용 기술

Django	DRF	PostgreSQL	Docker
JWT	OAuth		